Unit – 4

Why Python?

Python is a high-level, interpreted, general-purpose programming language. It is widely used because of its simple syntax, readability, and versatility.

Applications of Python:

1). Artificial Intelligence (AI)

- Used to develop intelligent systems.
- Used in machine learning and deep learning.
- Used in chatbots and virtual assistants.
- **Example:** ChatGPT, voice assistants.

2). Data Science

- Used to collect, process, and analyze large amounts of data.
- Used for data visualization and statistical analysis.
- Helps in predicting future trends.
- **Libraries:** NumPy, Pandas, Matplotlib.

3). Web Development

- Used to develop dynamic websites and web applications.
- Used for server-side programming.
- Helps in database connectivity and backend development.
- **Frameworks:** Django, Flask.

4). Automation

- Used to automate repetitive tasks.
- Used for file handling and data entry.
- Used for sending automated emails and web scraping.
- Helps save time and reduce human errors.

Variables

A variable is a name used to store data in a program. In Python, a variable is created when a value is assigned to it. No separate declaration is required.

Syntax:

variable_name = value

Example:

```
a = 10
b = 20
name = "Ankit"
print(a)
print(b)
print(name)
```

Output:

```
10
20
Ankit
```

Rules for Naming Variables:

1. **1).** A variable name must start with a letter or underscore (_).
2. **2).** A variable name cannot start with a digit.
3. **3).** A variable name can contain letters, digits and underscores.
4. **4).** Spaces are not allowed in variable names.
5. **5).** Python keywords cannot be used as variable names.
6. **6).** Variable names are case-sensitive.

Valid Variable Names:

```
name = "Ankit"
age = 20
_student = "Rahul"
total_marks = 450
```

Invalid Variable Names:

```
2name = "Ankit"
my name = "Ankit"
class = 10
```

Data Types

A data type defines the type of value that a variable can store. Python provides different built-in data types to store different kinds of data.

Data Type	Description	Example
int	Stores integer numbers	a = 10
float	Stores decimal numbers	a = 10.5
complex	Stores complex numbers	a = 2+3j
str	Stores text or strings	a = "Python"
bool	Stores True or False	a = True
list	Stores multiple values	a = [10, 20, 30]
tuple	Stores multiple values	a = (10, 20, 30)
set	Stores unique elements	a = {10, 20, 30}
dict	Stores key-value pairs	a = {"name": "Ankit"}

Example Program:

```
a = 10
b = 10.5
c = 2 + 3j
d = "Python"
e = True
f = [10, 20, 30]
g = (10, 20, 30)
h = {10, 20, 30}
i = {"name": "Ankit"}
print(type(a))
print(type(b))
print(type(c))
print(type(d))
print(type(e))
print(type(f))
```

```
print(type(g))

print(type(h))

print(type(i))
```

Output:

```
<class 'int'>

<class 'float'>

<class 'complex'>

<class 'str'>

<class 'bool'>

<class 'list'>

<class 'tuple'>

<class 'set'>

<class 'dict'>
```

print() Function

The print() function is used to display output on the screen. It is one of the most commonly used built-in functions in Python.

Syntax:

```
print(value)
```

Example 1: Printing a String

```
print("Hello World")
```

Output:

```
Hello World
```

Example 2: Printing a Number

```
print(100)
```

Output:

```
100
```

Example 3: Printing Multiple Values

```
a = 10

b = 20
```

print(a, b)

Output:

10 20

Example 4: Printing Variables with Messages

name = "Ankit"

age = 20

print("My name is", name)

print("My age is", age)

Output:

My name is Ankit

My age is 20

input() Function

The input() function is used to accept data from the user through the keyboard.

By default, the input() function returns the entered value as a string.

Syntax:

variable_name = input("Message")

Example 1: Taking String Input

name = input("Enter your name: ")print("Welcome", name)

Output:

Enter your name: Ankit

Welcome Ankit

Example 2: Taking Integer Input

To accept an integer value, we use the int() function with input().

age = int(input("Enter your age: "))

print("Your age is", age)

Example 3: Taking Float Input

To accept a decimal value, we use the float() function.

price = float(input("Enter price: "))

print("Price =", price)

Example 4: Addition of Two Numbers

```
a = int(input("Enter first number: "))

b = int(input("Enter second number: "))

sum = a + bprint("Sum =", sum)
```

Output:

Enter first number: 10

Enter second number: 20

Sum = 30

Operators in Python

An operator is a special symbol or keyword used to perform operations on variables and values.

Python supports the following types of operators:

1. **1).** Arithmetic Operators
2. **2).** Relational or Comparison Operators
3. **3).** Logical Operators
4. **4).** Assignment Operators
5. **5).** Bitwise Operators
6. **6).** Membership Operators
7. **7).** Identity Operators

A. Arithmetic Operators

Arithmetic operators are used to perform mathematical operations such as addition, subtraction, multiplication and division.

Operator	Name	Description	Example	Result
+	Addition	Adds two values	10 + 5	15
-	Subtraction	Subtracts two values	10 - 5	5
*	Multiplication	Multiplies two values	10 * 5	50
/	Division	Returns quotient	10 / 3	3.333...
%	Modulus	Returns remainder	10 % 3	1
//	Floor Division	Returns quotient rounded down	10 // 3	3
**	Exponentiation	Calculates power	2 ** 3	8

Example Program:

```
a = 10
b = 3
print("Addition =", a + b)
print("Subtraction =", a - b)
print("Multiplication =", a * b)
print("Division =", a / b)
print("Modulus =", a % b)
print("Floor Division =", a // b)
print("Power =", a ** b)
```

Output:

Addition = 13

Subtraction = 7

Multiplication = 30

Division = 3.3333333333333335

Modulus = 1

Floor Division = 3

Power = 1000

B. Relational or Comparison Operators

Relational operators are used to compare two values. They return either True or False.

Operator	Name	Example	Result
==	Equal to	10 == 10	True
!=	Not equal to	10 != 5	True
>	Greater than	10 > 5	True
<	Less than	5 < 10	True
>=	Greater than or equal to	10 >= 10	True

$a = 10$

$b = 5$

```
print(a < b)
```

```
print(a >= b)
print(a <= b)
```

Output:

```
False
True
False
```

C. Logical Operators

Logical operators are used to combine two or more conditions. They return Boolean results.

Python has three logical operators:

Operator	Description	Example
and	Returns True if both conditions are True	10 > 5 and 20 > 10
or	Returns True if at least one condition is True	10 < 5 or 20 > 10
not	Reverses the Boolean result	not(10 > 5)

Example Program:

```
a = 10
b = 20
print(a > 5 and b > 10)
print(a > 15 or b > 10)
print(not(a > 5))
```

Output:

```
True
True
False
```

D. Assignment Operators

Assignment operators are used to assign values to variables and update their values.

Operator	Example	Equivalent Expression
=	$a = 10$	Assigns 10 to a
+=	$a += 5$	a = a + 5

$-=$	$a -= 5$	a = a - 5
$*=$	$a *= 5$	a = a * 5
$/=$	$a /= 5$	a = a / 5
$\% =$	$a \% = 5$	a = a % 5
$//=$	$a //= 5$	a = a // 5
$**=$	$a **= 2$	a = a ** 2

Example Program:

a = 10

$a += 5$

print(a)

$a -= 3$

print(a)

$a *= 2$

print(a)

Output:

15

12

24

Difference Between = and ==

= (Assignment Operator)	**== (Comparison Operator)**
Used to assign a value to a variable.	Used to compare two values.
It assigns a value.	It returns True or False.
Example: a = 10	Example: a == 10

Example:

a = 10

print(a == 10)

Output:

True

Control Flow in Python

Control flow refers to the order in which statements are executed in a program. It controls the execution of statements based on conditions and repetitions.

Main control flow statements:

1). Conditional Statements – if, if-else, elif.

2). Looping Statements – for, while.

if Statement

The if statement executes a block of code only when the given condition is True.

Syntax:

```
if condition:
    statement
```

Example:

```
age = 20
if age >= 18:
    print("Eligible to vote")
```

Output:

Eligible to vote

if-else Statement

The if-else statement executes one block when the condition is True and another block when the condition is False.

Syntax:

```
if condition:
    statement1
else:
    statement2
```

Example:

```
num = 10
if num % 2 == 0:
    print("Even Number")
else:
```

```
    print("Odd Number")
```

Output:

Even Number

if-elif-else Statement

It is used to check multiple conditions in a program.

Syntax:

```
if condition1:
    statement1
elif condition2:
    statement2
else:
    statement3
```

Example:

```
marks = 75
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Fail")
```

Output:

Grade B

Nested if Statement

An if statement inside another if statement is called a nested if statement.

Example:

```
age = 20
```

```
if age >= 18:
    if age >= 21:
        print("Eligible")
    else:
        print("Not eligible")
else:
    print("Under age")
```

Output:

Not eligible

for Loop in Python

A for loop is used to execute a block of statements repeatedly for each element in a sequence such as a list, tuple or string.

Syntax:

```
for variable in sequence:
    statements
```

Example 1:

```
for i in range(1, 4):
print(i)
```

Output:

1

2

3

range() Function

The range() function generates a sequence of numbers and is commonly used with the for loop.

Syntax	**Description**
range(5)	Generates 0 to 4
range(1, 6)	Generates 1 to 5
range(1, 10, 2)	Generates 1, 3, 5, 7, 9

Example 2: Sum of First 10 Natural Numbers

```
sum = 0
for i in range(1, 11):
    sum = sum + i
print("Sum =", sum)
```

Output:

Sum = 55

while Loop in Python

A while loop executes a block of statements repeatedly as long as the given condition is True.

Syntax:

```
while condition:
    statements
```

Example 1:

```
i = 1
while i <= 3:
    print(i)
    i = i + 1
```

Output:

1

2

3

Example 2: Sum of First 10 Natural Numbers

```
i = 1
sum = 0
while i <= 10:
    sum = sum + i
    i = i + 1
print("Sum =", sum)
```

Output: Sum = 55

Functions in Python

A function is a block of reusable code that performs a specific task. It helps to reduce code repetition and makes a program easier to understand and maintain.

Python provides two types of functions:

1). **Built-in Functions:** Functions already available in Python, such as print(), input(), len() and type().

2). **User-defined Functions:** Functions created by the programmer using the def keyword.

def Keyword

The def keyword is used to define or create a user-defined function in Python.

Syntax:

```
def function_name(parameters):
    statements
```

Example:

```
def greet():
    print("Hello, Welcome to Python")
greet()
```

Output:

Hello, Welcome to Python

Explanation:

- def is used to define a function.
- greet is the function name.
- () contains parameters.
- The function body contains statements.
- greet() calls the function.

Function with Parameters and Arguments

Parameters are variables defined in a function definition, whereas arguments are actual values passed to the function during the function call.

Example:

```
def add(a, b):
    print("Sum =", a + b)
```

```
add(10, 20)
```

Output:

```
Sum = 30
```

Here, a and b are parameters, whereas 10 and 20 are arguments.

return Statement

The return statement is used to send a result back from a function to the calling statement. It also terminates the function execution.

Syntax:

```
def function_name():
    return value
```

Example:

```
def add(a, b):
    return a + b
result = add(10, 20)
print("Sum =", result)
```

Output:

```
Sum = 30
```

Explanation:

- The function receives two arguments.
- It calculates their sum.
- The return statement sends the result back.
- The result is stored in the variable result.

Note: If a function does not contain a return statement, Python automatically returns None.

Default Arguments

Default arguments are the values assigned to function parameters during function definition. If the user does not provide an argument, the default value is used.

Syntax:

```
def function_name(parameter=value):
    statements
```

Example:

```
def greet(name="Ankit"):
    print("Hello", name)
greet()
greet("Rahul")
```

Output:

Hello Ankit

Hello Rahul

Explanation:

- In the first function call, no argument is passed, so the default value "Ankit" is used.
- In the second function call, "Rahul" is passed, so it replaces the default value.

Example of Default Arguments with Multiple Parameters

```
def student(name, course="B.Tech"):
    print("Name:", name)
    print("Course:", course)
student("Ankit")
student("Rahul", "BCA")
```

Output:

Name: Ankit

Course: B.Tech

Name: Rahul

Course: BCA